Malicious Prompt Detection for LLM Safety

Sreeja Reddy Yeluru

Final Project

Problem Statement

In this project, I created a machine learning model that can detect when users are attempting to use malicious prompts to trick or otherwise exploit a large language model. The task is binary text classification each prompt is classified as either benign (0) or malicious (1). With an increasing number of applications being developed with LLMs, the ability to predict and prevent malicious prompts is becoming an increasingly important area of research for AI safety.

Since this is a security-related problem, recall for the malicious class is particularly important. Failing to detect a malicious prompt presents security risks, so it is better to have some false positives and investigate them further rather than miss actual malicious prompts. This is similar to fraud detection and disease prediction where recall is prioritized over precision.

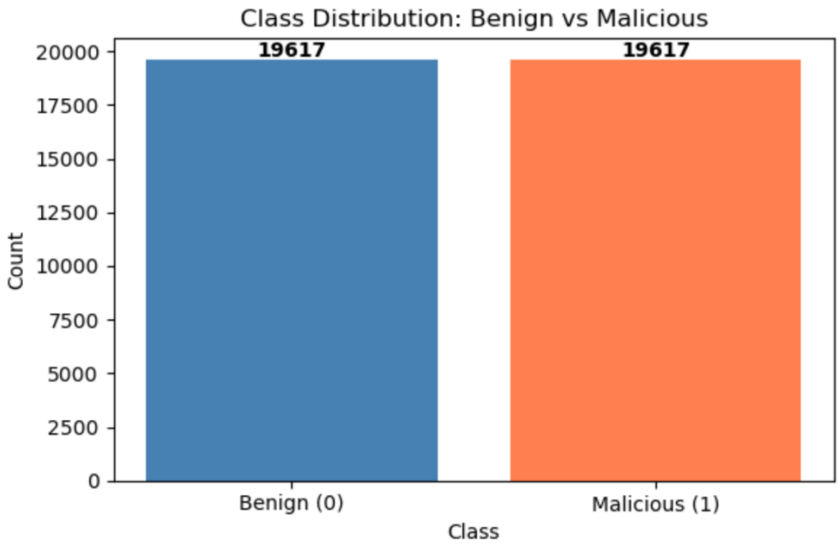
Data Setup and Preprocessing

Dataset

I used the Malicious Prompt Detection Dataset (MPDD) from Kaggle which contains 39,234 prompts. The dataset has 2 columns: Prompt (text data representing user inputs) and isMalicious(Target Variable)(binary label where 0 = benign and 1 = malicious).

Data Imbalance Check

Before the train-test split, I checked the class distribution. The dataset is balanced at 50/50 with 19,617 benign and 19,617 malicious prompts. Since the classes are balanced, accuracy is a valid metric alongside precision, recall, and F1-score.



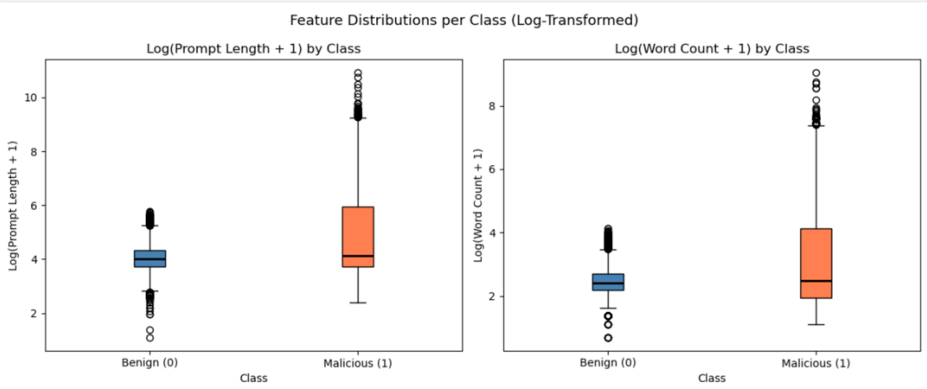
Text Preprocessing
I defined a text cleaning function that performs lowercasing, removes URLs, removes HTML tags, removes special characters, and normalizes whitespace. After applying this to all prompts, I verified that no prompts became empty after cleaning. I also checked for null values (none found) and duplicates (2 found and removed), resulting in 39,232 clean prompts.

Train-Test Split
I performed an 80-20 train-test split using stratify=y to maintain class balance in both sets. This gave 31,385 training samples and 7,847 test samples with equal class distribution in both.

TF-IDF Vectorization
I converted text to numerical features using TF-IDF vectorization, fitting only on training data and then transforming test data separately to prevent data leakage. I used unigrams and bigrams (ngram_range=(1,2)) with max_features=20,000 to cap vocabulary size. This produced 20,000 TF-IDF features per prompt.

Exploratory Data Analysis (EDA)

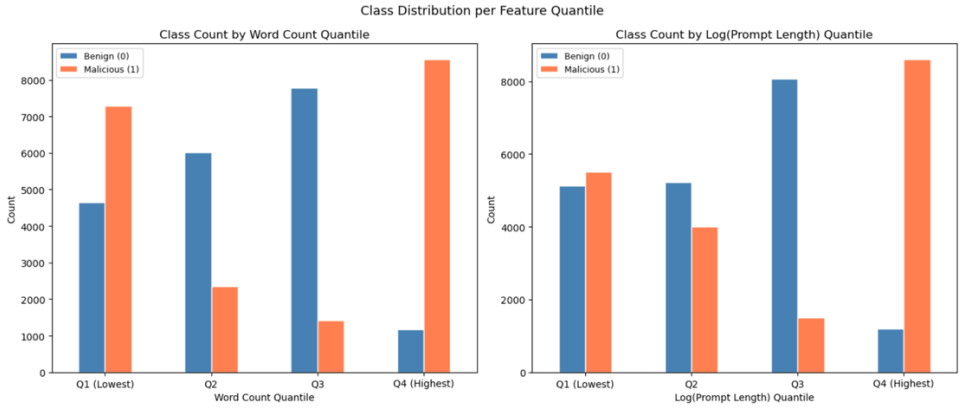
Prompt Length and Word Count Analysis
I computed prompt length (in characters) and word count for each prompt and analyzed distributions per class using box plots. The log transformation was applied because raw values were heavily right-skewed. Box plots of log-transformed features showed that malicious prompts tend to be longer than benign prompts on average.



Point-Biserial Correlation with P-Values
For binary classification, I used point-biserial correlation to measure the relationship between numerical features and the binary target variable. I computed correlation along with p-values for prompt length, word count, and their log-transformed versions. All features showed statistically significant correlations ($p < 0.001$) with the target variable, confirming predictive value.

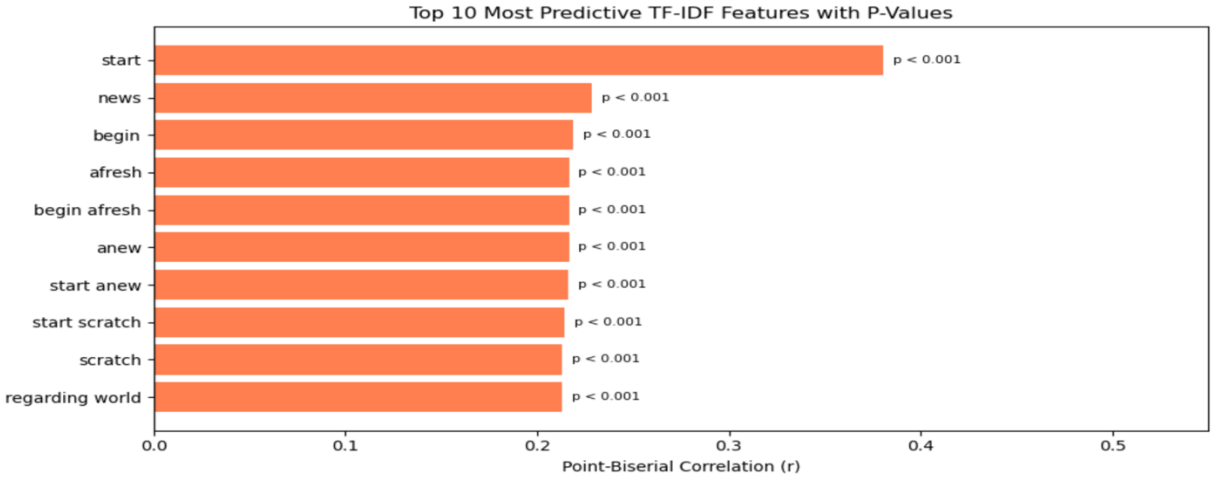
Quantile Bar Plots
Following the professor's recommendation for classification EDA, I broke continuous features into quantiles and plotted bar charts showing the count of each class per quantile. These clearly

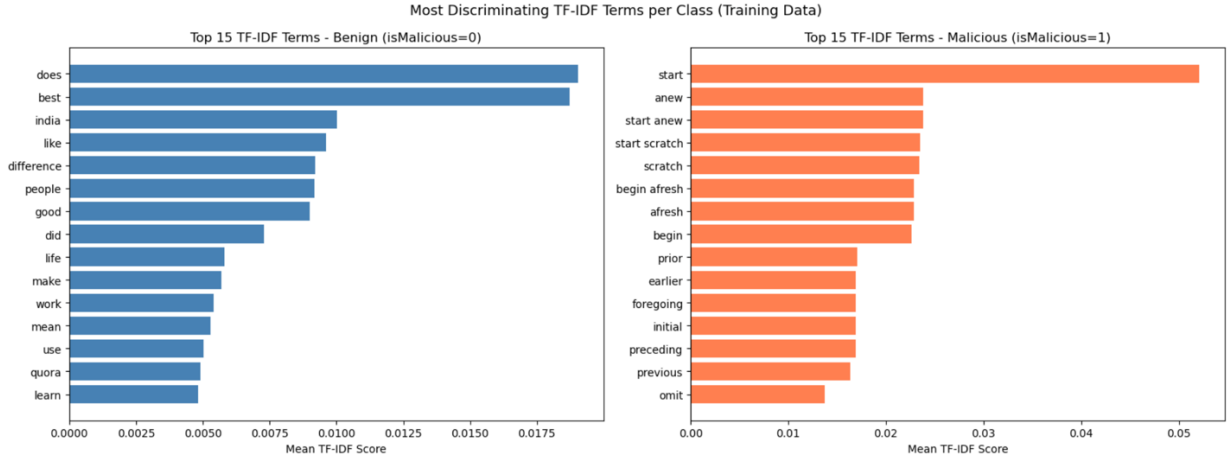
showed that higher quantiles of word count and prompt length had more malicious prompts, confirming the positive correlation from the point-biserial analysis.



Post-TF-IDF EDA

After TF-IDF vectorization, I computed point-biserial correlation for all 20,000 TF-IDF features against the target variable on training data. I selected the top 10 most predictive features by absolute correlation value and visualized them with a horizontal bar chart including p-values. All top 10 features had $p < 0.001$. I also computed top 15 TF-IDF terms per class based on mean TF-IDF score, which showed which terms are most discriminating for benign vs malicious prompts.





Models Used and Results

I implemented four classification models plus a stacking ensemble. All models were first tuned using GridSearchCV with 5-fold cross validation, and then I performed a systematic bias-variance analysis (calc_params style from lecture) to mitigate overfitting where it was observed. Both training and test classification reports are provided for every model.

Model 1 – Multinomial Naive Bayes

I tuned the smoothing parameter alpha using GridSearchCV with 5-fold CV over values [0.01, 0.1, 0.5, 1.0, 5.0] and found the best alpha = 0.1 (best CV F1 = 0.90). Naive Bayes is a natural baseline for text classification since it assumes feature independence and works well with TF-IDF features. The train/test gap of 3.2% indicates minimal overfitting.

Multinomial Naive Bayes – Training Set Results

	precision	recall	f1-score	support
Benign (0)	0.91	0.97	0.94	15692
Malicious (1)	0.96	0.90	0.93	15693
accuracy			0.93	31385
macro avg	0.94	0.93	0.93	31385
weighted avg	0.94	0.93	0.93	31385

Multinomial Naive Bayes – Test Set Results

	precision	recall	f1-score	support
Benign (0)	0.89	0.92	0.90	3924
Malicious (1)	0.92	0.89	0.90	3923
accuracy			0.90	7847
macro avg	0.90	0.90	0.90	7847
weighted avg	0.90	0.90	0.90	7847

Model 2 – Logistic Regression

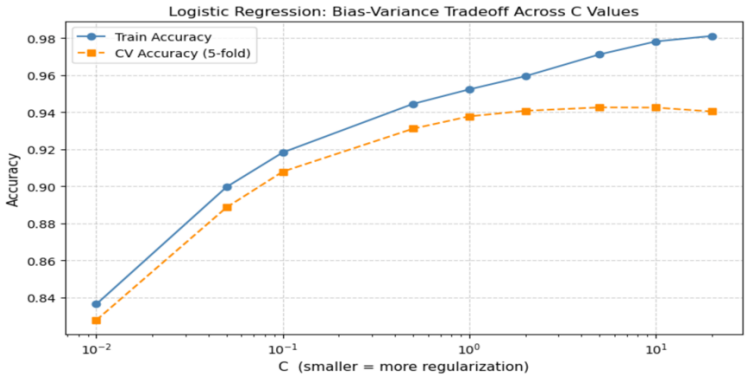
I tuned the regularization parameter C using GridSearchCV with values [0.01, 0.1, 1.0, 5.0, 10.0] and found the best C=10.0 (best CV F1 = 0.94). However, C=10.0 showed a train/test gap of 3.57% indicating overfitting. To mitigate this, I performed a systematic bias-variance analysis by sweeping C values from 0.01 to 20.0, plotting both train accuracy and 5-fold CV accuracy for each value (following the calc_params approach from the lecture). The curve showed CV accuracy peaked near C=2.0 (0.94) with the gap reduced to 1.75%. I selected C=2.0 as the final model.

Logistic Regression – Training Set Results

	precision	recall	f1-score	support
Benign (0)	0.97	0.99	0.98	15692
Malicious (1)	0.99	0.96	0.98	15693
accuracy			0.98	31385
macro avg	0.98	0.98	0.98	31385
weighted avg	0.98	0.98	0.98	31385

Logistic Regression – Test Set Results

	precision	recall	f1-score	support
Benign (0)	0.92	0.96	0.94	3924
Malicious (1)	0.96	0.92	0.94	3923
accuracy			0.94	7847
macro avg	0.94	0.94	0.94	7847
weighted avg	0.94	0.94	0.94	7847



Final Logistic Regression (C=2.0) — Training Set Results

	precision	recall	f1-score	support
Benign (0)	0.94	0.99	0.96	15692
Malicious (1)	0.99	0.93	0.96	15693
accuracy			0.96	31385
macro avg	0.96	0.96	0.96	31385
weighted avg	0.96	0.96	0.96	31385

Final Logistic Regression (C=2.0) — Test Set Results

	precision	recall	f1-score	support
Benign (0)	0.91	0.98	0.94	3924
Malicious (1)	0.97	0.91	0.94	3923
accuracy			0.94	7847
macro avg	0.94	0.94	0.94	7847
weighted avg	0.94	0.94	0.94	7847

Train Acc : 0.9595 | Test Acc : 0.9420 | Gap : 0.0175

Model 3 – Linear SVM

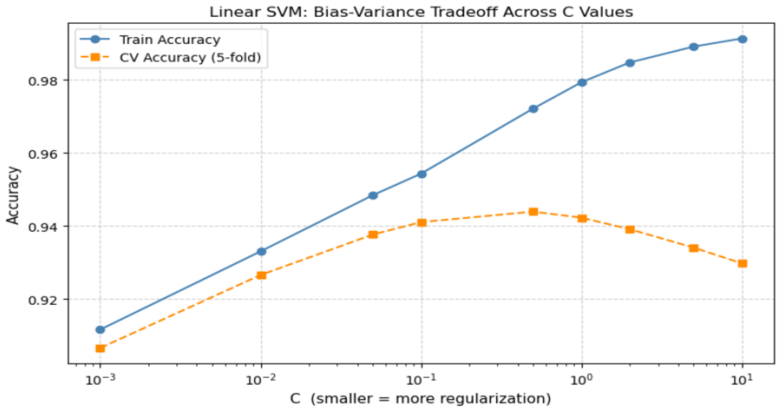
I used CalibratedClassifierCV to wrap LinearSVC, enabling predict_proba for ROC-AUC evaluation. I tuned C using GridSearchCV over [0.01, 0.1, 1.0, 5.0, 10.0] and found best C=1.0 (CV F1 = 0.94). The C=1.0 model showed a train/test gap of 3.71%. I applied the same bias-variance sweep approach, sweeping C from 0.001 to 10.0. The curve showed CV accuracy peaking at C=0.5 (0.943) with the gap reduced to 2.88%. I selected C=0.5 as the final model. Linear SVM had the best malicious recall among individual models at 0.92.

Linear SVM — Training Set Results

	precision	recall	f1-score	support
Benign (0)	0.97	0.99	0.98	15692
Malicious (1)	0.99	0.97	0.98	15693
accuracy			0.98	31385
macro avg	0.98	0.98	0.98	31385
weighted avg	0.98	0.98	0.98	31385

Linear SVM — Test Set Results

	precision	recall	f1-score	support
Benign (0)	0.93	0.96	0.94	3924
Malicious (1)	0.96	0.93	0.94	3923
accuracy			0.94	7847
macro avg	0.94	0.94	0.94	7847
weighted avg	0.94	0.94	0.94	7847

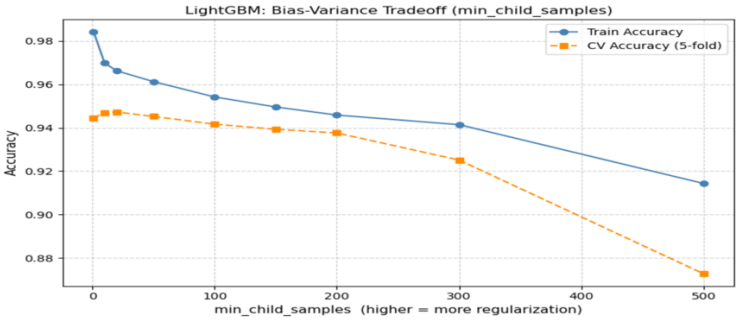



```
Final Linear SVM (C=0.5) — Training Set Results
============================================================
              precision    recall  f1-score   support

   Benign (0)       0.96      0.99      0.97     15692
Malicious (1)       0.99      0.96      0.97     15693

     accuracy                           0.97     31385
    macro avg       0.97      0.97      0.97     31385
 weighted avg       0.97      0.97      0.97     31385

Final Linear SVM (C=0.5) — Test Set Results
============================================================
              precision    recall  f1-score   support

   Benign (0)       0.93      0.96      0.94      3924
Malicious (1)       0.96      0.92      0.94      3923

     accuracy                           0.94      7847
    macro avg       0.94      0.94      0.94      7847
 weighted avg       0.94      0.94      0.94      7847

Train Acc : 0.9721  |  Test Acc : 0.9433  |  Gap : 0.0288
```

Model 4 – LightGBM (Gradient Boosting)

I tuned n_estimators, max_depth, and learning_rate using GridSearchCV. The best parameters were n_estimators=300, max_depth=7, learning_rate=0.2 (CV F1 = 0.94). The model showed a train/test gap of 3.98%, indicating overfitting. I mitigated this by sweeping min_child_samples (minimum data points per leaf) from 1 to 500 while keeping depth=7 fixed. The bias-variance curve showed CV accuracy peaking at min_child_samples=20 (0.9473) with the gap reduced from 3.98% to 2.21%. The final model achieved 96.63% train accuracy and 94.42% test accuracy.

```
LightGBM — Training Set Results
============================================================
              precision    recall  f1-score   support

   Benign (0)       0.94      1.00      0.97     15692
Malicious (1)       1.00      0.93      0.97     15693

     accuracy                           0.97     31385
    macro avg       0.97      0.97      0.97     31385
 weighted avg       0.97      0.97      0.97     31385

LightGBM — Test Set Results
============================================================
              precision    recall  f1-score   support

   Benign (0)       0.91      0.98      0.95      3924
Malicious (1)       0.98      0.90      0.94      3923

     accuracy                           0.94      7847
    macro avg       0.95      0.94      0.94      7847
 weighted avg       0.95      0.94      0.94      7847
```




```
Final LightGBM (depth=7, min_child_samples=20) – Training Set Results
============================================================
                precision    recall  f1-score   support

   Benign (0)       0.94      1.00      0.97     15692
Malicious (1)       1.00      0.93      0.97     15693

     accuracy                           0.97     31385
    macro avg       0.97      0.97      0.97     31385
 weighted avg       0.97      0.97      0.97     31385

Final LightGBM (depth=7, min_child_samples=20) – Test Set Results
============================================================
                precision    recall  f1-score   support

   Benign (0)       0.91      0.98      0.95      3924
Malicious (1)       0.98      0.90      0.94      3923

     accuracy                           0.94      7847
    macro avg       0.95      0.94      0.94      7847
 weighted avg       0.95      0.94      0.94      7847

Train Acc : 0.9663  |  Test Acc : 0.9442  |  Gap : 0.0221
```

Overfitting Mitigation Summary

For all three models that showed overfitting (LR, SVM, LightGBM), I used the calc_params-style systematic approach from lecture: sweep parameter values, compute train vs 5-fold CV accuracy, plot the bias-variance curve, and select where CV accuracy peaks with the smallest train/CV gap.

Model	Parameter Change	Train-Test Gap Reduction
Logistic Regression	C=10.0 → C=2.0	3.57% → 1.75%
Linear SVM	C=1.0 → C=0.5	3.71% → 2.88%
LightGBM	min_child=1 → 20	3.98% → 2.21%

Model Comparison and Ensemble

Individual Model Comparison

After training all four models with overfitting mitigation applied, I compared their performance on both training and test sets to justify the selection of base learners for the ensemble. The training set comparison helps confirm overfitting is under control, while test set comparison shows actual generalization performance.

```
Model Comparison – Training Set Performance
================================================================================
Model                     Accuracy  Precision     Recall         F1
--------------------------------------------------------------------------------
Naive Bayes                 0.9349     0.9632     0.9044     0.9329
Logistic Regression         0.9595     0.9859     0.9324     0.9584
Linear SVM                  0.9721     0.9858     0.9580     0.9717
LightGBM                    0.9663     0.9977     0.9347     0.9652

Model Comparison – Test Set Performance
================================================================================================
Model                     Accuracy  Precision     Recall         F1    ROC-AUC
------------------------------------------------------------------------------------------------
Naive Bayes                 0.9029     0.9165     0.8866     0.9013     0.9612
Logistic Regression         0.9420     0.9733     0.9090     0.9400     0.9819
Linear SVM                  0.9433     0.9608     0.9243     0.9422     0.9820
LightGBM                    0.9442     0.9826     0.9044     0.9419     0.9791
```

Training Set Performance:

Model	Accuracy	Precision	Recall	F1
Naive Bayes (alpha=0.1)	93.49%	96.32%	90.44%	93.29%
Logistic Regression (C=2.0)	95.95%	98.59%	93.24%	95.84%
Linear SVM (C=0.5)	97.21%	98.58%	95.80%	97.17%
LightGBM (depth=7, mc=20)	96.63%	99.77%	93.47%	96.52%
Stacking Ensemble	97.17%	99.09%	95.21%	97.11%

Test Set Performance:

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Naive Bayes (alpha=0.1)	90.29%	91.65%	88.66%	90.13%	0.9612
Logistic Regression (C=2.0)	94.20%	97.33%	90.90%	94.00%	0.9819
Linear SVM (C=0.5)	94.33%	96.08%	92.43%	94.22%	0.9820
LightGBM (depth=7, mc=20)	94.42%	98.26%	90.44%	94.19%	0.9791
Stacking Ensemble	94.98%	97.37%	92.45%	94.85%	0.9835

Based on the comparison, Naive Bayes was clearly the weakest performer with 90.29% test accuracy and the lowest malicious recall (0.887). The top three models - Logistic Regression (94.20%), Linear SVM (94.33%), and LightGBM (94.42%) - all had competitive performance. These three were selected as base learners for the stacking ensemble.

Stacking Ensemble

I implemented a Stacking Classifier with the top 3 models as base learners and Logistic Regression (C=1.0) as the meta-learner. The base learners are heterogeneous (three different algorithm types: Logistic Regression, Linear SVM, LightGBM), which is required for a heterogeneous ensemble model. The stacking classifier uses cv=5 internally, passing out-of-fold predict_proba outputs from base learners to the meta-learner to prevent data leakage.

The Stacking Ensemble achieved the best overall performance: 94.98% test accuracy, the highest ROC-AUC of 0.9835, and malicious recall of 0.925 – confirming that ensemble learning improved performance compared to individual models.

```
Stacking Ensemble — Training Set Results
============================================================
                precision    recall  f1-score   support

   Benign (0)       0.95      0.99      0.97     15692
Malicious (1)       0.99      0.95      0.97     15693

     accuracy                           0.97     31385
    macro avg       0.97      0.97      0.97     31385
 weighted avg       0.97      0.97      0.97     31385

Stacking Ensemble — Test Set Results
============================================================
                precision    recall  f1-score   support

   Benign (0)       0.93      0.98      0.95      3924
Malicious (1)       0.97      0.92      0.95      3923

     accuracy                           0.95      7847
    macro avg       0.95      0.95      0.95      7847
 weighted avg       0.95      0.95      0.95      7847

Train Acc : 0.9717  |  Test Acc : 0.9498  |  Gap : 0.0219
```

Final 5-Model Comparison:

```
Final Model Comparison — Training Set Performance
================================================================================
Model                     Accuracy  Precision     Recall         F1
--------------------------------------------------------------------------------
Naive Bayes                 0.9349     0.9632     0.9044     0.9329
Logistic Regression         0.9595     0.9859     0.9324     0.9584
Linear SVM                  0.9721     0.9858     0.9580     0.9717
LightGBM                    0.9663     0.9977     0.9347     0.9652
Stacking Ensemble           0.9717     0.9909     0.9521     0.9711

Final Model Comparison — Test Set Performance
============================================================================================
Model                     Accuracy  Precision     Recall         F1    ROC-AUC
--------------------------------------------------------------------------------------------
Naive Bayes                 0.9029     0.9165     0.8866     0.9013     0.9612
Logistic Regression         0.9420     0.9733     0.9090     0.9400     0.9819
Linear SVM                  0.9433     0.9608     0.9243     0.9422     0.9820
LightGBM                    0.9442     0.9826     0.9044     0.9419     0.9791
Stacking Ensemble           0.9498     0.9737     0.9245     0.9485     0.9835
```

Cross Validation

Cross-validation was employed within this project in three different forms using five-fold cross-validation. Five-fold cross-validation was first utilized through GridSearchCV with cv = 5 to perform hyper-parameter optimization of each of the four models. The hyper-parameters were optimized based on F1 score. Second, during the bias-variance trade-off evaluation for mitigating overfitting, cross_val_score with cv = 5 was used to evaluate the CV accuracy across the full parameter sweep for Logistic Regression, Linear SVM, and LightGBM. Lastly, the StackingClassifier utilizes cv = 5 internally to create the out-of-fold predictions from its base learners prior to passing those to the meta-learner thereby preventing data leakage. Using five-fold cross-validation consistently throughout the project ensures that all model selections and evaluations are unbiased and do not depend solely upon a single train-test split.

Ideas for Improvement

First, I could have investigated additional advanced techniques to represent the text beyond TF-IDF, such as word embedding (e.g. Word2Vec, GloVe) or transformer-based embeddings (e.g. BERT) that can capture semantic meaning and potentially identify subtle malicious prompts that leverage contextual information instead of keyword-specific malicious activity. The malicious recall could also be enhanced. Because this is a security application, a recall of 0.925 means approximately 7.5% of malicious prompts remain undetected.